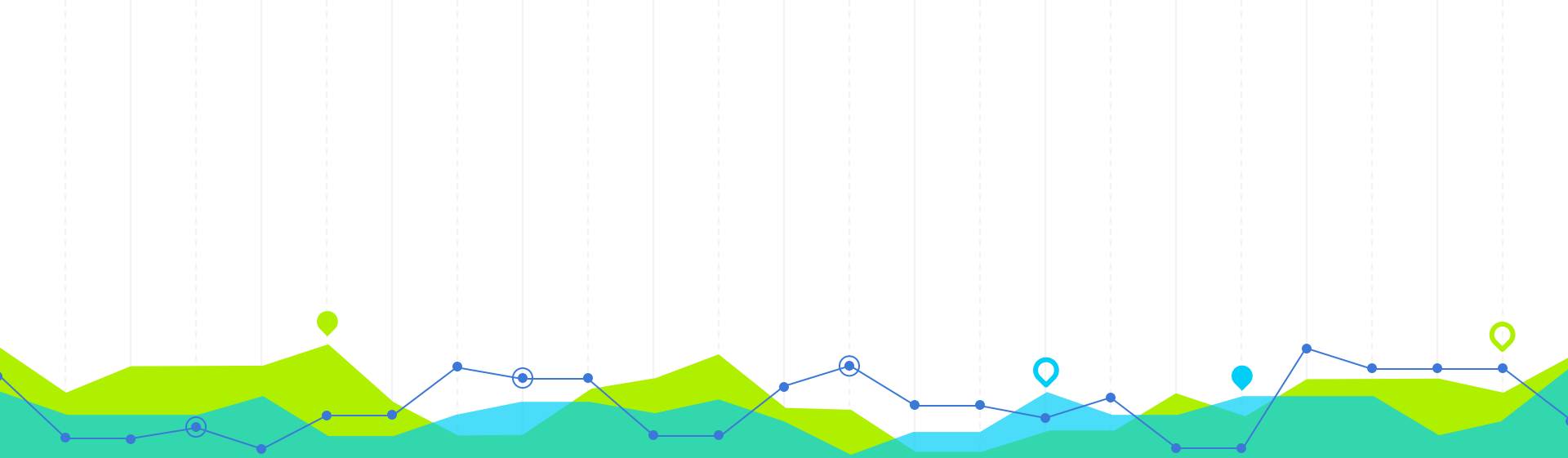
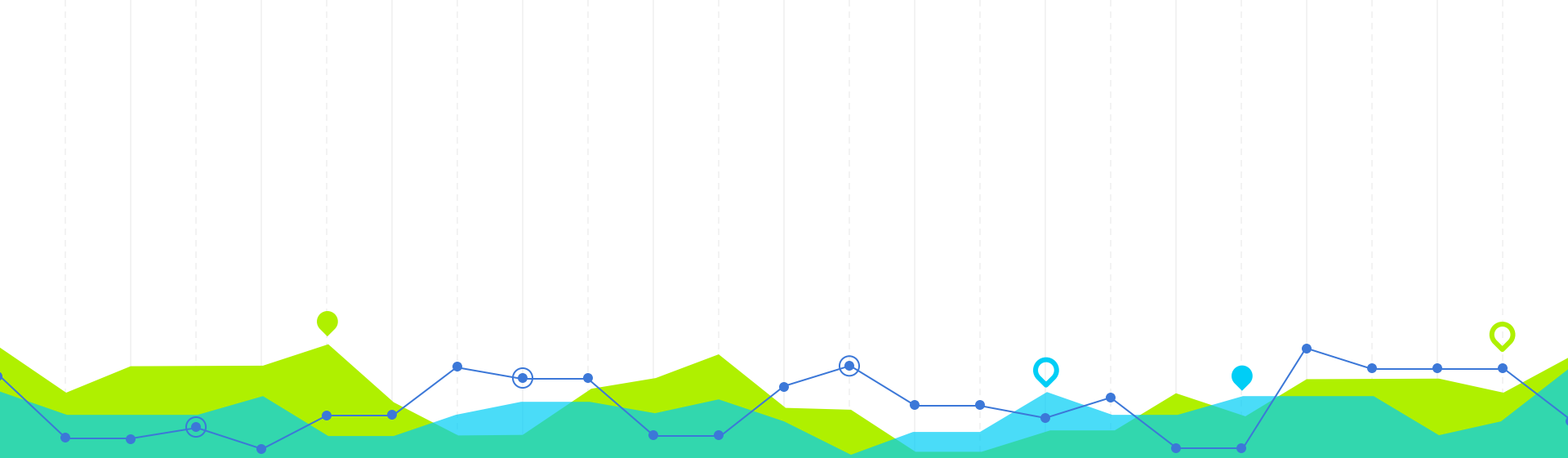




Coder en Python

pour la production



CI / CD

Intégration Continue & Déploiement Continu

Le programme

13:30 - 15:00	CI / CD Intégration Continue & Déploiement Continu
15:00 - 15:15	
15:15 - 16:45	Introduction à Streamlit et présentation du projet

Intégration Continue & Déploiement Continu

Objectifs

- Comprendre un pipeline CI/CD
- Apprendre à créer et déployer une application web interactive avec Streamlit

Programme

1. Introduction à la CI/CD
2. L'Intégration Continue (CI)
3. Le Déploiement Continu (CD)
4. Introduction à Streamlit

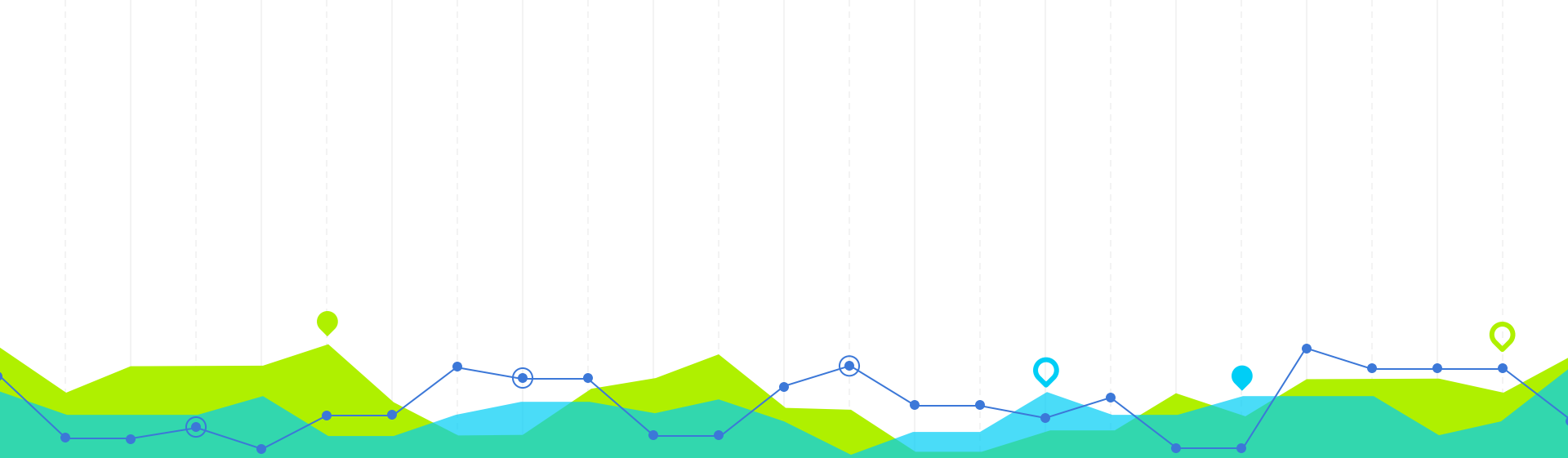


Objectif

CI/CD

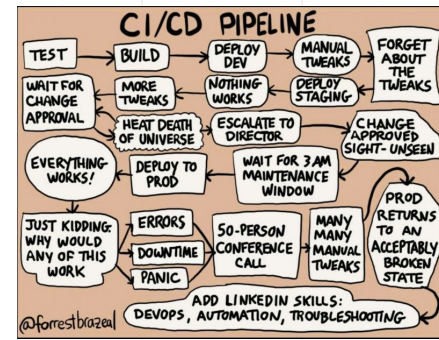
Une bonne pipeline de CI/CD
= mise en production relax 😊





1. Introduction à la CI/CD

Définitions



Intégration continue (CI)

Permet d'intégrer, tester, et valider le code **automatiquement** et **régulièrement** (ex : après chaque push)

L'objectif est d'identifier et de résoudre les problèmes le plus tôt possible, réduisant ainsi les coûts de la correction des bugs.

Livraison continue (Continuous Delivery)

Le code est **automatiquement préparé** pour un **déploiement en production** après avoir passé les étapes de CI.

Cela signifie que le code est toujours dans un état où il peut être déployé en production à tout moment.

Toutefois, le processus de déploiement en lui-même nécessite une intervention humaine.

Déploiement continu (Continuous Deployment)

Extension de la livraison continue. Ici, chaque changement de code qui passe toutes les étapes du processus de build et de test est automatiquement déployé en production => processus de mise en production entièrement automatisé.

Pourquoi utiliser un pipeline de CI/CD ?



- **Rapidité de mise en prod** : fonctionnalités et correctifs sont déployés plus rapidement, tout en évitant les régressions
- **Qualité améliorée** : l'automatisation des tests permet de détecter les problèmes et les résoudre plus tôt dans le cycle de développement.
- **Meilleure collaboration** : les équipes de développement et d'exploitation travaillent plus étroitement ensemble
- **Feedback rapide** : les développeurs reçoivent rapidement des retours sur leur travail, ce qui leur permet d'ajuster en conséquence.

Assure la qualité, la robustesse et la fiabilité du code, surtout lorsqu'il s'agit de code de production.





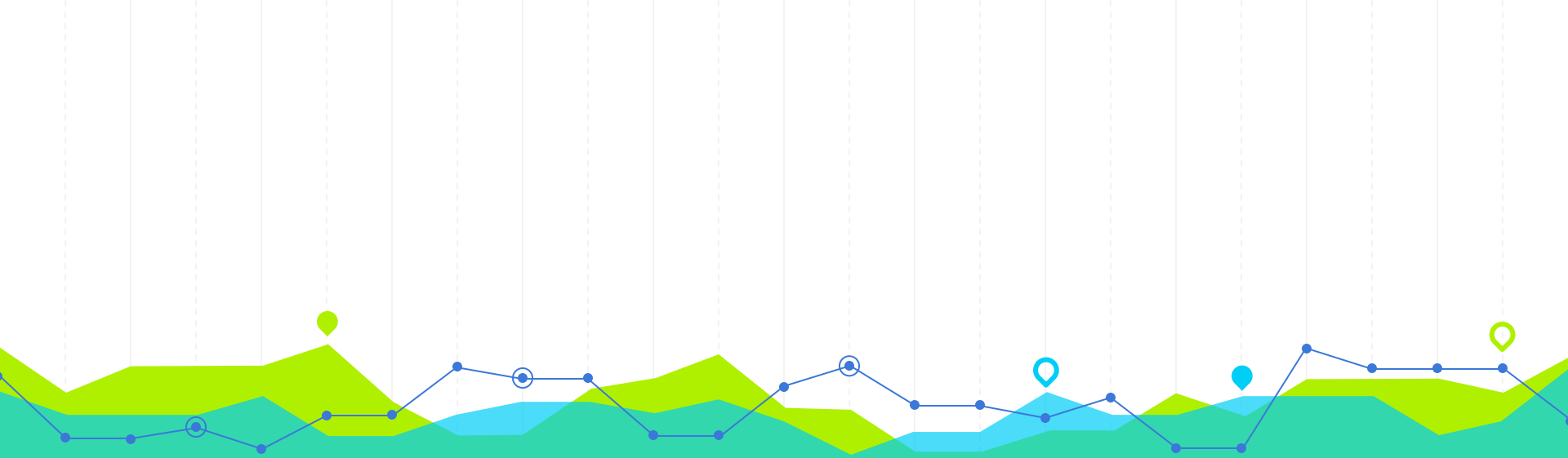
Comparatifs d'outils de CI/CD

Solution CI/CD	Avantages	Inconvénients
Jenkins	Flexible : configurable avec une multitude de plugins pour différents outils et langages Open Source : gratuit et peut être hébergé sur votre propre serveur Grande communauté et beaucoup de ressources disponibles.	Configuration difficile à configurer Mises à jour qui peut parfois casser des configurations existantes Interface utilisateur datée
Travis CI	Intégration avec GitHub Facilité de configuration des fichiers <code>.travis.yml</code> pour la configuration Plateformes multiples : prend en charge de nombreuses langues et plateformes Gratuit pour les projets open source	Coût : les projets privés peuvent devenir coûteux Limité à GitHub



Comparatifs d'outils de CI/CD

Solution CI/CD	Avantages	Inconvénients
GitHub Actions	Intégré à GitHub Flexibilité : permet des workflows complexes Gratuit pour les projets open source : minutes gratuites chaque mois	Jeune Limité à GitHub
GitLab CI	Intégration avec GitLab Prise en charge native de Docker et Kubernetes Auto DevOps : fonctionnalités intégrées qui automatisent de nombreux aspects du DevOps Business model : Il y a une version gratuite disponible, et tarification compétitive pour les versions plus avancées	Courbe d'apprentissage complexe Performances parfois limité avec les runners GitLab CI



1. Intégration Continue (CI)



Qu'est ce que la CI ?

Objectifs : intégrer **automatiquement** et **régulièrement** les changements apportés au code, détecter le plus tôt possible les erreurs éventuelles et **assurer la qualité du code**

Pourquoi la CI est importante pour le code de production ?

- Une erreur identifiée **tôt** dans le cycle de développement est généralement moins coûteuse à corriger
- **Automatisation des tests** : cela garantit que chaque changement introduit n'entraîne pas de régressions.
- **Qualité du code** : adoption des normes et des pratiques de codage => code de qualité et plus maintenable
- Les développeurs savent que leurs contributions sont **vérifiées** contre le code existant
- Les équipes sont **plus confiance** en la qualité de leur code, sachant qu'il est régulièrement testé et validé
- Bonne **documentation** du code, via des commentaires appropriés et à la génération automatique de la documentation
- **Livraison continue** : La CI est le premier pas vers la mise en place d'une livraison continue (CD, pour "Continuous Delivery"), où chaque changement intégré peut être livré aux utilisateurs en production de manière fluide et régulière



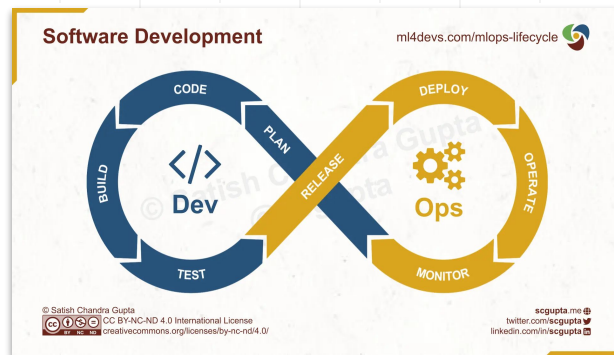
Intégration Continue (CI)

Plan : discutez des **objectifs** à atteindre et des **données nécessaires** pour évaluer les **performances** d'un modèle de ML dans le monde réel.

Code : concevoir et développer le logiciel, le produit ou l'application **de bout en bout (API, Webapp, pipeline, etc.)** pour être sur la même longueur d'onde entre les équipes.

Build : alimente l'**intégration continue** des différents développements et leurs mises à disposition (bibliothèques, SDK, Image Docker, binaire, etc.)

Test : tests unitaires, fonctionnels, d'intégration, de performance, de bout en bout, etc.
automatisez-les autant que possible.



Intégration Continue (CI)

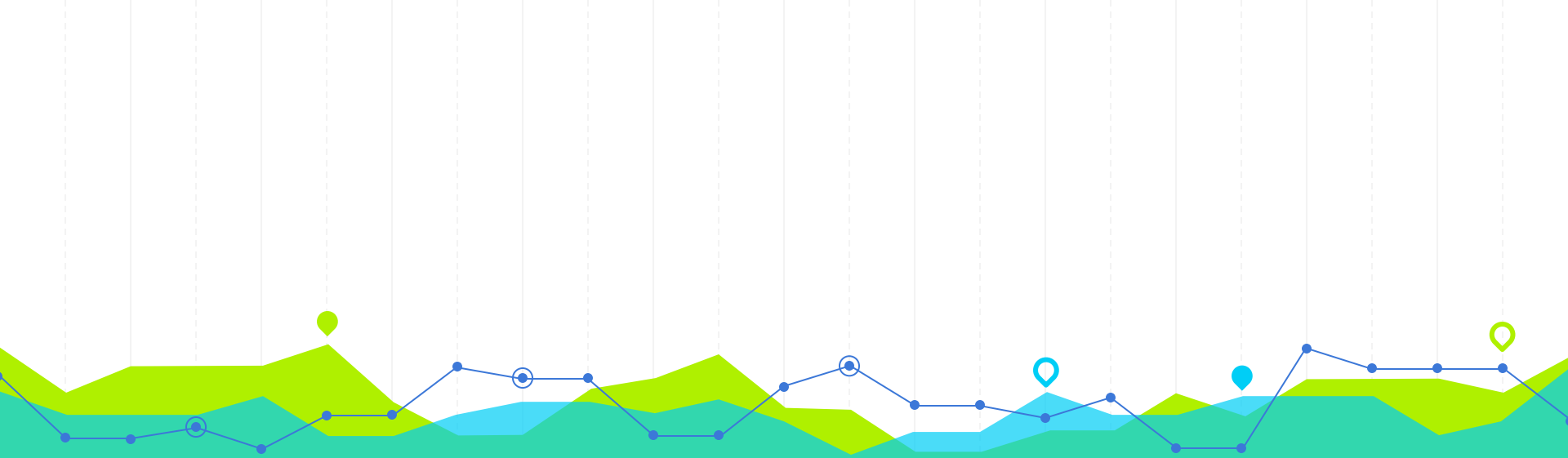


La CI démarre lorsque des modifications sont poussées dans une branche via une PR (Pull Request).

Un exemple de CI :

1. **Récupération de la dernière version du code (Checkout)**
2. **Configuration de l'environnement (Setup)** - création d'un environnement virtuel (avec uv par ex)
3. **Exécution des tests (Testing)** - `pytest`
4. **Analyse de la couverture de code testée (Coverage Analysis)** - `coverage.py`
5. **Analyse statique du code (Linting et Static Analysis)** - `flake8`, `pylint`, ou `mypy`

Notifications : en cas d'échec de l'une des étapes ci-dessus, informez l'équipe ou le développeur concerné (emails, Slack, etc.)



2. Déploiement Continu

Déploiement Continu (CD)



Objectif : livrer automatiquement les modifications du code dans un environnement de production

Pourquoi la CD est importante pour le code de production ?

1. **Livraisons plus fréquentes et plus prévisibles** de fonctionnalités, corrections et de mises à jour
2. **Qualité constante** en **automatisant le déploiement** et en éliminant la variabilité humaine => moins d'erreurs
3. **Diminution des risques et flexibilité** avec des livraisons plus **petites** et plus **fréquentes**, plus facile à déboguer
4. Les équipes n'ont pas à se soucier des déploiements manuels et peuvent se concentrer sur les nouvelles fonctionnalités
5. **Réduction des coûts** en évitant les erreurs de déploiement manuel et en réduisant les bugs post-déploiement

Il est essentiel d'avoir une **suite solide de tests automatisés** et des **environnements de déploiement** bien configurés.

Un pipeline de CD offre **d'énormes avantages** en termes d'**efficacité**, de **qualité** et de **rapidité de mise en production**.



Les environnements de développements



Environnement de développement (Dev)

- Les **nouvelles fonctionnalités** et les **corrections de bogues** sont développées et testées en interne ici
- Il s'agit généralement de la **machine locale** d'un développeur
- Utilise souvent des **données factices** ou un **petit échantillon** des données réelles

Environnement de qualification (Qualif)

- Tests de qualité sont effectués pour s'assurer que les fonctionnalités **répondent aux spécifications**.

Environnement de test (Test)

- Pour des **tests plus approfondis** : tests **d'intégration**, des tests de **charge**, des tests de **sécurité**, etc.
- Une **base de données** de test est utilisée qui est distincte de la production.



Les environnements de développements



Environnement de pré-production (Pre-prod)

- **Réplique** de l'environnement de production, ce qui permet de tester les mises à jour finales dans des **conditions similaires** à la production

Environnement de staging (Staging)

- **Juste avant** le déploiement en production
- Les tests de **dernière minute** et la validation finale sont effectués

Environnement de production (Production) :

- Les fonctionnalités sont **déployées en production**
- L'environnement de production est **accessible aux utilisateurs finaux**

Environnements supplémentaires :

- **QA (Assurance Qualité)** : utilisé spécifiquement par les équipes de test pour effectuer des tests manuels ou automatisés.
- **Environnement de CI** : les tests sont automatiquement exécutés ici chaque fois qu'un code est poussé vers une branche ou une PR

Chaque transition d'un environnement à l'autre est idéalement être automatisée via l'intégration continue et du déploiement continu (CI/CD).



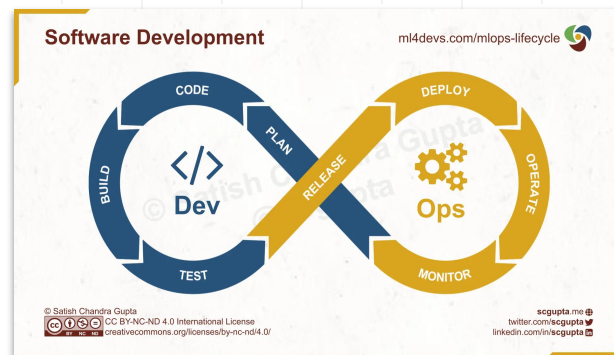
Déploiement Continu (CD)

Release : si les tests passent, on met à disposition les **artefacts** : le **code**, les **modèles** et ses **metadata** doivent être **versionnés** et **sauvegardés** automatiquement.

Deploy : choisir les **artefacts** à partir du repo docker ou du **model registry** et les **déployer sur l'infrastructure** de production (via AWS SageMaker, PyTorch Serve, etc.)

Operate : vous pouvez décider de votre **stratégie de déploiement** (déploiement blue-green, canary, rollback, etc.). Kubernetes et KubeFlow sont des outils couramment utilisés.

Monitor : **surveiller** en permanence l'état des services, les erreurs, les latences, les prédictions du modèle, les valeurs aberrantes, etc. **Rollback** ou **réentraîner** si besoin.



Déploiement Continu (CD)



La CD est en continuité de la CI.

Un exemple de CD :

1. **Création d'artefacts**

Empaquetage du code pour la distribution : création d'une Wheel, d'un exécutable avec **PyInstaller**, ou image Docker

- **Constructions et déploiements conditionnels (Conditional Builds & Deployments)**

Dans certains cas, on peut vouloir construire des packages ou déployer du code sous certaines conditions

Ex : build et pousser un wheel Python vers PyPI **seulement** si la branche est **master** ou si un tag a été ajouté

2. **Déploiement sur un environnement de staging/pré-production :**

- Permet de **valider le comportement de l'application** dans un environnement qui **imite la production**.

- Exécution de **tests automatisés** pour s'assurer que l'application fonctionne comme prévu sur cet environnement.

3. **Validation manuelle (si nécessaire) :**

Parfois, certaines validations ne peuvent pas être automatisées. Une équipe de QA peut valider manuellement.

Déploiement Continu (CD)



4. Déploiement en production

- Déploiement de l'application sur les serveurs/instances/containers de production.
- Si on utilise une approche de déploiement **blue-green**, le nouveau code est déployé **parallèlement** à l'ancien, et une fois que tout est confirmé comme fonctionnant, le trafic est basculé sur le nouveau déploiement.

5. Monitoring & Rollback :

- Surveillance continue de l'application en production à l'aide d'outils de monitoring et d'alerting.
- Si un problème est détecté, il est essentiel d'avoir une stratégie pour revenir à une version précédente

6. Feedback :

- Collecte de retours de l'équipe de développement, de l'équipe opérationnelle, et des utilisateurs pour améliorer le processus et corriger les problèmes éventuels.

En plus des étapes ci-dessus, des aspects tels que la sécurité (par exemple, les scans de vulnérabilité), l'observabilité (logs, traçage, etc.) aide à garantir un déploiement réussi et sans heurts.

Approche "shift-left" : s'occuper de ces aspects le plus tôt possible dans le cycle de développement,



Exemple de .gitlab-ci.yml

```
stages:      # Définit les différentes étapes de CI/CD
  - lint
  - test
  - deploy   # Déploiement

variables:   # Variables globales disponibles dans toutes les étapes
  PYTHON_VERSION "3.9"
  PIP_CACHE_DIR  "$CI_PROJECT_DIR/.cache/pip"

cache:       # Cache pour les dépendances pip afin d'accélérer les pipelines
  paths:
    - .cache/pip

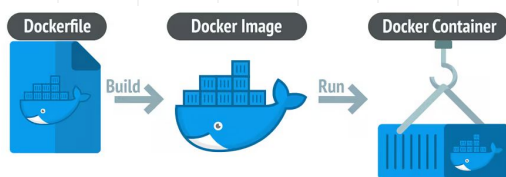
before_script # Actions à effectuer avant chaque job
  - python --version
  - pip install --upgrade pip

# Étape 1: Lint
lint:
  stage: lint
  image: python:$PYTHON_VERSION
  script:
    - pip install flake8      # Installe l'outil d'analyse statique
    - flake8 .                # Exécute flake8 pour analyser le code
  only:
    - merge_requests          # Exécute uniquement pour les MR
  allow_failure true          # Si le lint échoue, on ne bloque pas le pipeline

# Étape 2: Test
test:
  stage: test
  image: python:$PYTHON_VERSION
  script:
    - pip install -r requirements.txt  # Installe les dépendances
    - pip install pytest              # Installe pytest
    - pytest                          # Exécute les tests
  artifacts:
    paths:
      - tests/reports/               # Sauvegarde les rapports de tests (optionnel)
  only:
    - branches                       # Exécute les tests sur toutes les branches sauf 'master' ou 'main'
```

```
# Étape 3: Deploy
deploy_production
  stage: deploy
  image: python:$PYTHON_VERSION
  script:
    - echo "Deploying to production environment..."
    # Ajoute ici les étapes réelles de déploiement, comme ssh ou une commande kubectl
  only:
    - master                        # Le déploiement n'est déclenché que sur la branche master/main
  environment:
    name: production
    url: https://your-production-url.com

deploy_staging
  stage: deploy
  image: python:$PYTHON_VERSION
  script:
    - echo "Deploying to staging environment..."
    # Ajoute ici les étapes de déploiement pour staging
  only:
    - develop                       # Le déploiement sur staging se fait via la branche develop
  environment:
    name: staging
    url: https://your-staging-url.com
```



Docker

Docker est une plateforme de **conteneurisation** qui permet de développer, de tester et de déployer des applications dans des **conteneurs**.

Chaque conteneur est une **instance de l'image** (créé via un fichier DockerFile)

Les avantages de Docker :

- **Isolation** : chaque application est exécutée dans un conteneur isolé, ce qui évite les conflits de dépendances et les problèmes de compatibilité.
- **Portabilité** : les conteneurs Docker sont portables et peuvent être exécutés sur n'importe quelle machine qui prend en charge Docker.
- **Facilité de déploiement** : simplifie le déploiement des applications, car il suffit de déployer le conteneur Docker sur le serveur cible.
- **Rapidité** : permet de lancer rapidement des environnements d'exécution pour les applications, ce qui accélère les cycles de développement et de déploiement.

Docker est comme une boîte magique pour les logiciels qui permet de les emballer avec tout ce dont ils ont besoin pour fonctionner, facilitant ainsi leur déploiement et leur exécution sur n'importe quel ordinateur.



CI/CD + Docker



Voici une explication étape par étape de comment cela pourrait fonctionner:

Intégration Continue (CI)

1. Construction du conteneur Docker

Le **Dockerfile** construit une image de votre application (image de base, dépendances, fichiers, etc.)

2. Tests

Lancer un conteneur de cette image et exécuter vos tests à l'intérieur. Si les tests échouent, le pipeline est arrêté.

Déploiement Continu (CD)

1. Push de l'image Docker

Si vos tests passent, vous pouvez pousser votre image Docker vers un **registre** comme Docker Hub ou un registre privé

2. Déploiement

Déployer l'image dans un **environnement cible** (serveur de production, de pré-production ou de test). Les orchestrateurs comme **Kubernetes** peuvent faciliter ce processus.

CI/CD + Docker



Avantages de cette approche

1. **Reproductibilité et portabilité** : chaque conteneur garantit un **fonctionnement similaire** dans tout environnement.
2. **Isolation** : Docker isole votre application et ses dépendances du système hôte. **Pas de conflits** de dépendances.
3. **Facilité de déploiement** : simplifie le déploiement des applications, il suffit de **déployer le conteneur** Docker sur le serveur
4. **Accélère les cycles de dev / déploiement** en lançant rapidement des environnements d'exécution pour les applications
5. **Scalabilité** : Avec Docker et Kubernetes, il est facile de **mettre à l'échelle** votre application (**scaling horizontal**)

En combinant Python, Docker, et un pipeline CI/CD => assure une **livraison rapide, reproductible et fiable** des applications

Ce processus automatisé **réduit les erreurs humaines**, **accélère les cycles de livraison** et garantit que les **tests** sont toujours exécutés avant le déploiement.

Exemple de .gitlab-ci.yml avec Docker

```
stages:
  - build      # Construction de l'image Docker
  - test       # Tests unitaires
  - deploy     # Déploiement

variables:
  PYTHON_VERSION: "3.9"
  IMAGE_NAME: "${CI_REGISTRY_IMAGE}:${CI_COMMIT_REF_SLUG}"

# Utilisation du Docker-in-Docker pour permettre de construire des images Docker dans le pipeline
image: docker:20.10.16

services:
  - docker:20.10.16-dind # Docker daemon pour Docker-in-Docker (dind)

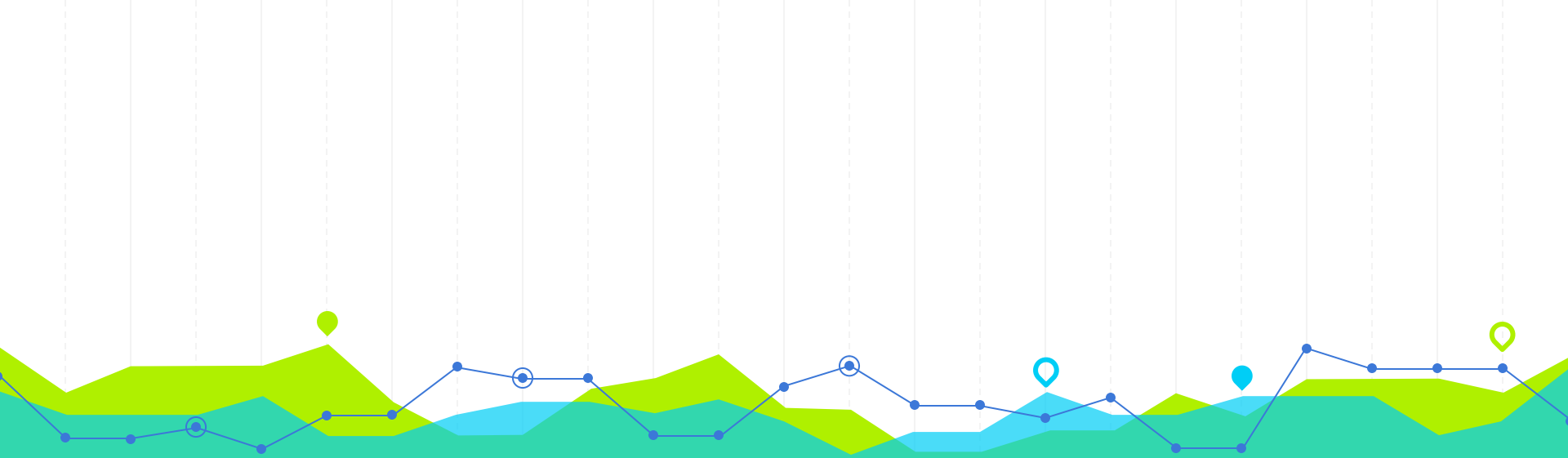
before_script:
  - docker login -u "${CI_REGISTRY_USER}" -p "${CI_REGISTRY_PASSWORD}" $CI_REGISTRY # Connexion au registre Docker

# Étape 1: Build Docker Image
build_image:
  stage: build
  script:
    - docker build -t $IMAGE_NAME . # Construit l'image Docker avec le Dockerfile présent à la racine
    - docker push $IMAGE_NAME       # Pousse l'image sur le registre GitLab
  only:
    - branches                       # Exécuter pour toutes les branches sauf master/main
  tags:
    - docker                        # Utiliser des runners GitLab tagués pour Docker

# Étape 2: Test
test:
  stage: test
  script:
    - docker pull $IMAGE_NAME        # Récupère l'image Docker construite lors de l'étape de build
    - docker run $IMAGE_NAME pytest # Exécute les tests avec pytest à l'intérieur du conteneur Docker
  only:
    - branches                       # Tester sur toutes les branches sauf master/main
  tags:
    - docker
```

```
# Étape 3: Deploy to Staging
deploy_staging:
  stage: deploy
  script:
    - docker pull $IMAGE_NAME        # Récupère l'image Docker construite
    - echo "Deploying to staging environment..."
    # Par exemple, utilisez kubectl, SSH ou Docker Compose pour déployer
  only:
    - develop                       # Le déploiement se fait sur la branche develop
  environment:
    name: staging
    url: https://staging.your-app.com
  tags:
    - docker

# Étape 4: Deploy to Production
deploy_production:
  stage: deploy
  script:
    - docker pull $IMAGE_NAME        # Récupère l'image Docker construite
    - echo "Deploying to production environment..."
    # Par exemple, utilisez kubectl, SSH ou Docker Compose pour déployer
  only:
    - master                       # Le déploiement se fait sur la branche master/main
  environment:
    name: production
    url: https://production.your-app.com
  tags:
    - docker
```



4. Introduction à Streamlit

Webapp



L'un des défis majeurs dans le domaine de la science des données est de **présenter** les résultats d'analyses de manière **interactive** et **conviviale**.

Heureusement, il existe les applications web (ou **Webapp**) !

Cas d'usage

- **Tableaux de bord de données** : visualisation dynamique de données avec des graphiques sur mesure
- **Prototypage de modèles ML** : interface simple pour tester des modèles
- **Outils d'analyse interne** : Création rapide d'outils pour des équipes techniques (style ETL par exemple)

Webapp



Il existe aujourd'hui **deux façons** de créer des webapp Python :

Depuis un jupyter : ipywidget et/ou ipyvuetify + voila + binder

- ipywidget : <https://ipywidgets.readthedocs.io/en/latest/examples/Widget%20List.html>
- ipyvuetify : <https://github.com/widgetti/ipyvuetify>
- Voilà : <https://voila-gallery.org/>
- Binder : <https://mybinder.org/>

Depuis un script python : avec l'une de ces solutions :

- Gradio : <https://www.gradio.app/demos>
- Dash : <https://dash.plotly.com/>
- Streamlit : <https://streamlit.io/gallery>
- Panel : <https://panel.holoviz.org/>
- Rio : <https://rio.dev/examples>



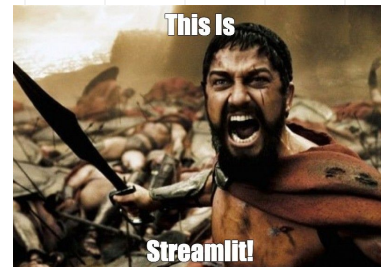
voilà



Streamlit

Streamlit est un framework Python open-source pour créer des **applications web interactives**.

- Idéal pour les **Data Scientists** : permet de transformer des scripts Python en webapp
- **Supporté** par Chrome, Firefox, Edge et Safari
- **Facile** à utiliser : aucune connaissance en HTML, CSS ou JavaScript n'est requise
- **Prototypage rapide** : créer des dashboards en **quelques lignes de code**
- **Interactivité** : ajout facile de widgets comme des boutons, des sliders, etc.
- **Visualisation** des données : compatible avec Matplotlib, Plotly, Seaborn, et plus
- **Déploiement** simple et rapide sur Streamlit Cloud ou autres



Streamlit

Installation de Streamlit

Pour installer Streamlit

```
$ uv pip install streamlit
```

Pour ouvrir une démo de webapp Streamlit :

```
$ uv run streamlit hello
```

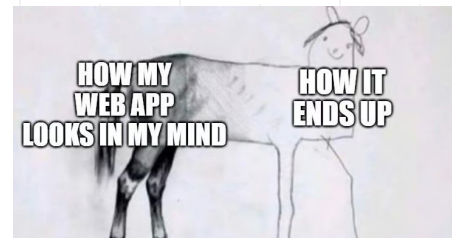
Pour lancer votre application Streamlit :

```
$ uv run streamlit run mon_app.py
```

....mais comment faire mon application Streamlit ?



Introduction à la création d'une première application Streamlit



Pour faire une webapp :

- Un seul fichier Python suffit.
- Aucune connaissance en HTML/CSS/JS nécessaire.
- Code s'exécute **en temps réel** avec Streamlit.

Les composants de base :

- Titres et textes :
 - `st.title()`: titre principal
 - `st.header()`: sous-titre
 - `st.text()`: texte simple
 - `st.markdown()`: texte avec formatage Markdown
- Afficher des tableaux de données :
 - `st.dataframe()`: tableau interactif
 - `st.table()`: tableau statique

Structure de base d'une app Streamlit :

```
import streamlit as st

st.title("Ma première application  
Streamlit")
st.header("Bienvenue sur l'application")
st.text("Ceci est un texte simple.")
```

Pour lancer l'application :
\$ streamlit run app.py

Exemple d'application interactive simple

```
import numpy as np

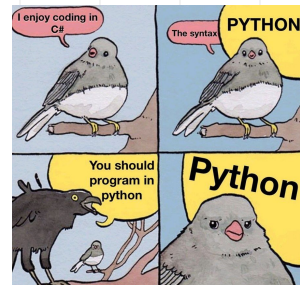
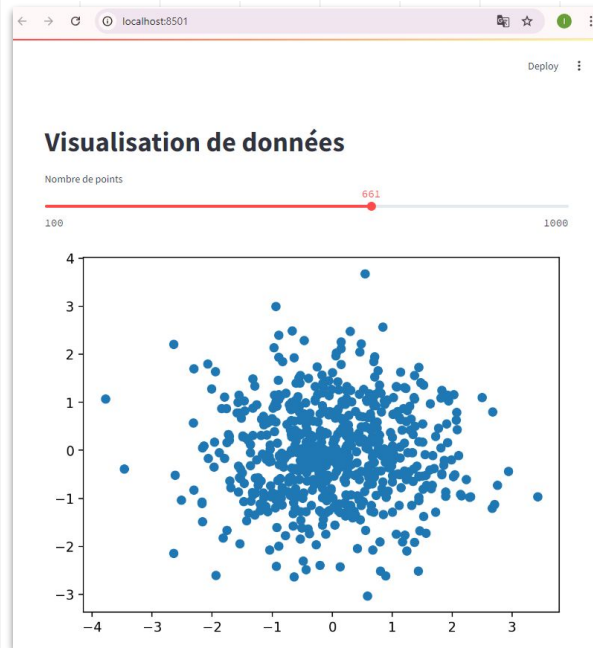
import matplotlib.pyplot as plt
import streamlit as st

st.title("Visualisation de données")

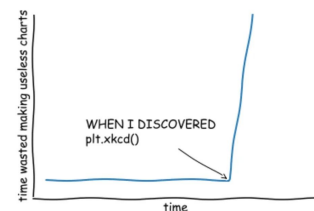
# Slider pour ajuster les données
points = st.slider("Nombre de points", 100, 1000, 500)

# Générer des données aléatoires
data = np.random.randn(points, 2)

# Afficher un graphique
fig, ax = plt.subplots()
ax.scatter(data[:, 0], data[:, 1])
st.pyplot(fig)
```



Streamlit et les graphiques



Streamlit prend en charge plusieurs bibliothèques graphiques différentes

- Matplotlib
- Vega Lite (graphiques en 2D)
- deck.gl (cartes et graphiques en 3D)
- graphiques qui sont « natifs » de Streamlit comme `st.line_chart` et `st.area_chart`
- Plotly
- Bokeh
- Altair
- etc.

Simple area charts

Display an area chart.

`st.area_chart(my_data_frame)`

Simple bar charts

Display a bar chart.

`st.bar_chart(my_data_frame)`

Simple line charts

Display a line chart.

`st.line_chart(my_data_frame)`

Simple scatter charts

Display a line chart.

`st.scatter_chart(my_data_frame)`

Scatterplots on maps

Display a map with points on it.

`st.map(my_data_frame)`

GraphViz

Display a graph using the `dagre-d3` library.

`st.graphviz_chart(my_graphviz_s)`

Matplotlib

Display a matplotlib.pyplot figure.

`st.pyplot(my_mpl_figure)`

Altair

Display a chart using the Altair library.

`st.altair_chart(my_altair_chart)`

Vega-Lite

Display a chart using the Vega-Lite library.

`st.vega_lite_chart(my_vega_lite)`

Plotly

Display an interactive Plotly chart.

`st.plotly_chart(my_plotly_chart)`

Bokeh

Display an interactive Bokeh chart.

`st.bokeh_chart(my_bokeh_chart)`

PyDeck

Display a chart using the PyDeck library.

`st.pydeck_chart(my_pydeck_chart)`

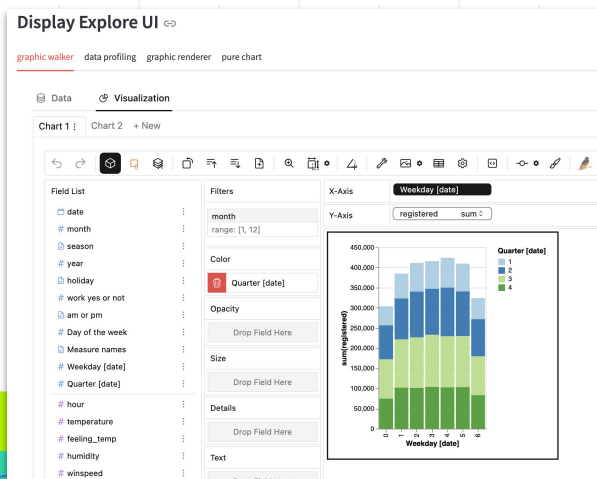
Les composants

Les composants sont des **modules tiers** qui étendent les possibilités de Streamlit

Construits par et pour la communauté

<https://streamlit.io/components>

<https://github.com/Kanaries/pygwalker>



Cloud Gallery Components Generative AI Community Docs Blog

Sign in Sign up

Components

Components are third-party modules that extend what's possible with Streamlit.

Built by creators, for the community

Want to build your own? Check out our [docs](#).

CATEGORIES

- All
- LLMs
- Widgets
- Charts
- Authentication
- Connections
- Images & video
- Audio
- Text
- Maps
- Dataframes
- Graphs
- Molecules & genes
- Code editors
- Page navigation
- Developer tools
- Integrations

Pygwalker

Kanaries

\$ pip install pygwalker

View demo →

HiPlot

facebookresearch

\$ pip install hiplot

View source →

Hub

pyscaffold

\$ pip install streamlit-hub

View source →

WebRTC

whitphx

\$ pip install streamlit-webrtc

View source →

Authenticator

mkhorasani

\$ pip install streamlit-authentic...

View source →

AgGrid

PablocFonseca

\$ pip install streamlit-aggrid

View source →

Gestion des entrées utilisateurs

Streamlit permet de **collecter des données** grâce à des **widgets interactifs**

Ces données peuvent être utilisées pour :

- **Personnaliser** l'application en fonction des utilisateurs
- **Interagir** avec les graphiques, modèles, ou autres éléments

Widgets interactifs principaux

- `st.text_input()` : saisie de texte
- `st.number_input()` : entrée de nombres
- `st.date_input()` : sélection d'une date
- `st.slider()` : curseur pour des valeurs numériques
- `st.selectbox()` : liste déroulante avec un seul choix
- `st.multiselect()` : liste déroulante avec choix multiples
- `st.button()` : bouton pour déclencher une action

Ici, l'utilisateur entre son nom et son âge, puis clique sur un bouton pour afficher un message.

```
name = st.text_input('Entrez votre nom')  
age = st.number_input('Entrez votre âge', min_value=0,  
max_value=120)  
  
if st.button('Soumettre'):  
    st.write(f'Bonjour {name}, vous avez {age} ans.')
```



Entrez votre nom

Martin

Entrez votre âge

33

Soumettre

Bonjour Martin, vous avez 33 ans.



Gestion des entrées utilisateurs

Les formulaires regroupent plusieurs widgets avec un bouton de **validation**

Permet de **réduire les interactions** inutiles avec l'interface

Conseils :

- **Utilisez des widgets** adaptés aux types de données
- **Regroupez les interactions** avec des formulaires pour plus de clarté
- **Vérifiez toujours les valeurs entrées** avant de les utiliser dans votre application

<https://docs.streamlit.io/develop/concepts/architecture/forms>

L'utilisateur entre ses informations et soumet le formulaire pour afficher les résultats :

```
with st.form('my_form') :  
    name = st.text_input('Nom')  
    age = st.slider('Âge', 0, 120)  
    submitted = st.form_submit_button('Soumettre')  
  
if submitted:  
    st.write(f'Nom : {name}, Âge : {age}')
```

Nom
Martin

Âge
0 33 120

Soumettre

Nom : Martin, Âge : 33

Gestion des entrées utilisateurs

Formulaire d'Informations Personnelles

Informations de base

Nom complet

Martin Prillard

Âge

33

Sexe

Homme

Coordonnées

Adresse email

prillard.martin@gmail.com

Numéro de téléphone

0836656565

Préférences

Méthode de contact préférée

☒ Email

☐ Téléphone

☒ Je souhaite recevoir la newsletter

Soumettre

Formulaire soumis avec succès !

Récapitulatif :

Nom : Martin Prillard

Âge : 33

Sexe : Homme

Email : prillard.martin@gmail.com

Téléphone : 0836656565

Préférence de contact : Email

Newsletter : Oui

```
import streamlit as st

# Titre de l'application
st.title("Formulaire d'Informations Personnelles")

# Définir un formulaire
with st.form("personal_info_form"):
    # Section 1 : Informations personnelles de base
    st.header("Informations de base")
    name = st.text_input("Nom complet")
    age = st.number_input("Âge", min_value=0,
max_value=120, step=1)
    gender = st.selectbox("Sexe", options=["Homme",
"Femme", "Autre"])

    # Section 2 : Coordonnées
    st.header("Coordonnées")
    email = st.text_input("Adresse email")
    phone = st.text_input("Numéro de téléphone")

    # Section 3 : Préférences
    st.header("Préférences")
    preferred_contact = st.radio("Méthode de contact
préférée", options=["Email", "Téléphone"])
    receive_newsletter = st.checkbox("Je souhaite
recevoir la newsletter")

    # Section 4 : Validation de la soumission
    submitted = st.form_submit_button("Soumettre")

# Vérification des entrées
if submitted:
    errors = []

    # Vérification du nom
    if not name:
        errors.append("Le nom ne peut pas être
vide.")
    elif len(name) < 2:
        errors.append("Le nom doit contenir au moins
2 caractères.")

    # Vérification de l'âge
    if age <= 0 or age > 120:
        errors.append("L'âge doit être compris entre
1 et 120 ans.")
```

```
# Vérification de l'adresse email
if "@" not in email or "." not in email.split("@")[-1]:
    errors.append("L'adresse email n'est pas valide.")

# Vérification du numéro de téléphone (exemple simple de
format)
if not phone.isdigit() or len(phone) < 8:
    errors.append("Le numéro de téléphone doit être composé de
chiffres et contenir au moins 8 caractères.")

# Affichage des erreurs ou des résultats
if errors:
    st.error("Erreur(s) dans le formulaire :")
    for error in errors:
        st.write(f"- {error}")
else:
    st.success("Formulaire soumis avec succès !")
    st.write("### Récapitulatif :")
    st.write(f"***Nom** : {name}")
    st.write(f"***Âge** : {age}")
    st.write(f"***Sexe** : {gender}")
    st.write(f"***Email** : {email}")
    st.write(f"***Téléphone** : {phone}")
    st.write(f"***Préférence de contact** :
{preferred_contact}")
    st.write(f"***Newsletter** : {'Oui' if receive_newsletter
else 'Non'}")
```

Sidebar



Streamlit facilite l'organisation de vos widgets dans un panneau latéral gauche avec `st.sidebar`.

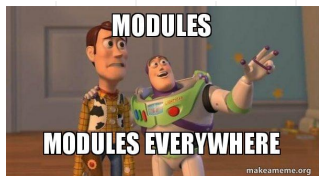
Chaque élément passé à `st.sidebar` est ajouté à gauche, ce qui permet aux utilisateurs de se concentrer sur le contenu de votre application tout en ayant accès aux contrôles de l'interface utilisateur.

```
import streamlit as st

# Add a selectbox to the sidebar:
add_selectbox = st.sidebar.selectbox(
    'How would you like to be contacted?',
    ('Email', 'Home phone', 'Mobile phone')
)

# Add a slider to the sidebar:
add_slider = st.sidebar.slider(
    'Select a range of values',
    0.0, 100.0, (25.0, 75.0)
)
```

A screenshot of a Streamlit application sidebar. It contains a selectbox titled 'How would you like to be contacted?' with 'Email' selected. Below it is a slider titled 'Select a range of values' with a range from 0.00 to 100.00, and the current selection is between 25.00 and 75.00.



Layout

Streamlit offre plusieurs autres moyens de contrôler la disposition de votre application.

`st.columns` vous permet de placer des widgets côte à côte :

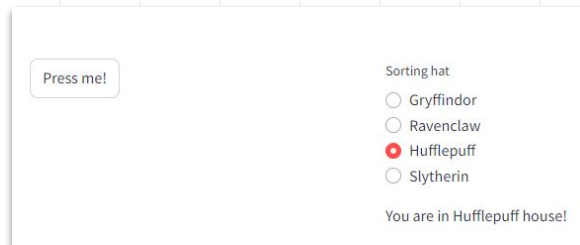
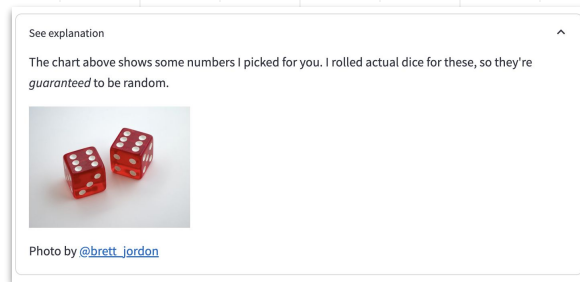
```
import streamlit as st

left_column, right_column = st.columns(2)
# You can use a column just like st.sidebar:
left_column.button('Press me!')

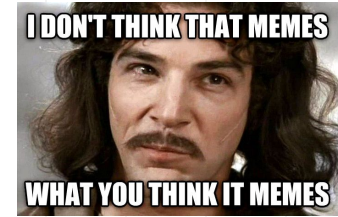
# Or even better, call Streamlit functions inside a "with" block:
with right_column:
    chosen = st.radio(
        'Sorting hat',
        ("Gryffindor", "Ravenclaw", "Hufflepuff", "Slytherin"))
    st.write(f"You are in {chosen} house!")
```



`st.expander` vous permet de conserver de l'espace en cachant le contenu :



Multipage apps



Lorsque les applications deviennent volumineuses, il devient utile de les **organiser en plusieurs pages**.

Cela facilite la gestion de l'application en tant que développeur et la navigation en tant qu'utilisateur.

Streamlit offre un moyen simple de créer des **applications multipages**.

<https://docs.streamlit.io/develop/concepts/multipage-apps>

```
▼ main_page.py

import streamlit as st

st.markdown("# Main page 🍌")
st.sidebar.markdown("# Main page 🍌")

▼ pages/page_2.py

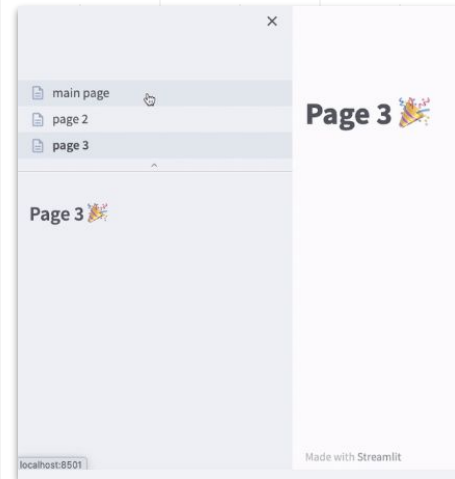
import streamlit as st

st.markdown("# Page 2 🍌")
st.sidebar.markdown("# Page 2 🍌")

▼ pages/page_3.py

import streamlit as st

st.markdown("# Page 3 🍌")
st.sidebar.markdown("# Page 3 🍌")
```



State Management



Une session est une **instance unique** de visualisation d'une application.

Chaque utilisateur (ou onglet) d'une application **dispose d'un état de session** lié à son affichage spécifique.

Streamlit **conserve cette session** au fur et à mesure que l'utilisateur interagit avec l'application.

Utilisez le dictionnaire `st.session_state` pour gérer et enregistrer les variables d'état dans vos applications, ce qui permet de conserver les données entre les interactions et les réexecutions de scripts

Exemple : stocker les sélections d'un utilisateur pour mettre à jour dynamiquement des graphiques ou des tables.

<https://docs.streamlit.io/develop/concepts/architecture/session-state>

```
import streamlit as st

if "counter" not in st.session_state:
    st.session_state.counter = 0

st.session_state.counter += 1

st.header(f"This page has run {st.session_state.counter} times.")
st.button("Run it again")
```



This page has run 7 times.

Run it again

Si l'utilisateur **actualise** la page de son navigateur ou **recharge l'URL** de l'application, son état de session est **réinitialisé** et il recommence avec une nouvelle session.

State Management



```
import streamlit as st

if "attendance" not in st.session_state:
    st.session_state.attendance = set()

def take_attendance():
    if st.session_state.name in st.session_state.attendance:
        st.info( f"{st.session_state.name } has already been counted.")
    else:
        st.session_state.attendance.add(st.session_state.name)

with st.form(key="my_form"):
    st.text_input( "Name", key="name")
    st.form_submit_button( "I'm here!", on_click=take_attendance)
```



Martin has already been counted.

Name

Martin

I'm here!

Les widgets gèrent eux-mêmes leur état, de sorte que vous n'aurez pas toujours besoin d'utiliser l'état de session !

Lorsqu'un utilisateur interagit avec un widget, l'ordre logique est le suivant :

1. Sa valeur dans `st.session_state` est mise à jour
2. La fonction de callback (le cas échéant) est exécutée
3. La page est réexécutée et la fonction du widget renvoie sa nouvelle valeur

Le Data flow de Streamlit

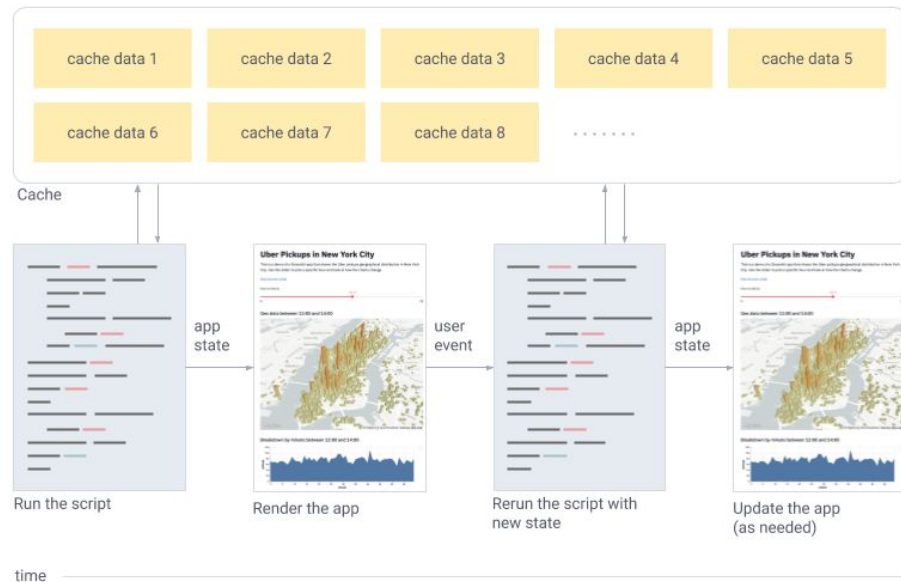


Chaque fois qu'un élément doit être mis à jour à l'écran, Streamlit **réexécute l'intégralité** de votre script Python, de haut en bas.

Cela peut se produire dans deux situations :

- Lorsque vous **modifiez le code** source de votre application
 - Chaque fois qu'un utilisateur **interagit** avec les widgets de l'application.
- Par exemple, en faisant glisser un curseur, en saisissant du texte dans une boîte de saisie ou en cliquant sur un bouton.

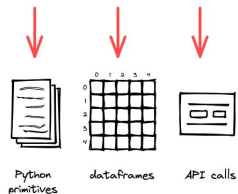
Lorsqu'une **fonction callback** (action sur un bouton par exemple) est transmise à un widget via le paramètre `on_change` (ou `on_click`), la fonction callback sera toujours **exécutée avant le reste** de votre script.



Le décorateur `@st.cache_data`, qui permet via un **cache**, aux développeurs d'ignorer certains calculs coûteux lorsque leurs applications sont réexécutées, joue un rôle important.

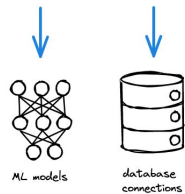
st.cache_data

anything you CAN store in a database

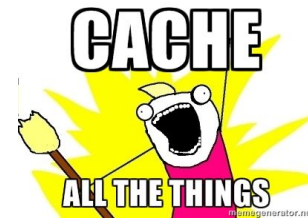


st.cache_resource

anything you CAN'T store in a database



Optimisation avec le cache



La mise en cache **améliore les performances** en évitant de recalculer des résultats coûteux pour les mêmes entrées :

- `st.cache_data` est recommandé pour les calculs renvoyant des données sérialisables (par exemple, str, int, float, DataFrame, dict, list), créant une copie protégée à chaque appel
- Pour les objets non sérialisables comme les modèles ML, utilisez `st.cache_resource`, qui partage l'objet entre sessions sans duplication, mais notez que les mutations affecteront toutes les réexecutions

<https://docs.streamlit.io/develop/concepts/architecture/caching>

```
@st.cache_data
```

```
def long_function(param1, param2):
```

```
    # Perform expensive computation here or
```

```
    # fetch data from the web here
```

```
    return data
```

```
@st.cache_resource
```

```
def init_model():
```

```
    # Return a global resource here
```

```
    return pipeline(
```

```
        "sentiment-analysis",
```

```
        model="distilbert-base-uncased-finetuned-sst-2
```

```
)
```

```
import streamlit as st
import pandas as pd
```

```
df = pd.read_csv("dir/file.csv")
```

```
@st.cache_data
```

```
def convert_df(df):
```

```
    return
```

```
df.to_csv(index=False).encode('utf-8')
```

```
csv = convert_df(df)
```

```
st.download_button(
```

```
    "Press to Download",
```

```
    csv,
```

```
    "file.csv",
```

```
    "text/csv",
```

```
    key='download-csv'
```

```
)
```

Déploiement d'une application Streamlit



Objectif : rendre votre application accessible en ligne à toutes et à tous !

Une solution : Streamlit Cloud

1. Créer un compte ou se connecter sur streamlit.io.
2. Assurez-vous que votre code est sur un dépôt **GitHub** public.
3. Depuis le tableau de bord de **Streamlit Cloud**, cliquer sur “**New app**”.
4. Sélectionner le dépôt GitHub contenant votre application.
5. Choisir le fichier Python principal (généralement `app.py`).
6. Cliquer sur “**Deploy**”.
7. Attendre que Streamlit Cloud construise et lance l'application.
8. Votre application est maintenant accessible via une URL publique.

Autres options : **Heroku**, **AWS**, **GCP**, **Azure**, **Docker**, **serveurs dédiés**

Déploiement d'une application Streamlit

AWS (Amazon Web Services)

- Installez AWS CLI et Elastic Beanstalk CLI (EB CLI)
- Créez un fichier `requirements.txt` pour les dépendances de votre application
- Dans le répertoire contenant votre application, initialisez un projet Elastic Beanstalk :
 - a. `eb init -p python-3.8 my-streamlit-app`
- Créez un environnement et déployez :
 - a. `eb create my-streamlit-env`
 - b. `eb deploy`
- Obtenez l'URL publique via la console Elastic Beanstalk ou la commande `eb open`

Heroku

- Ajouter un Procfile avec : `web: streamlit run app.py`
- Utiliser `git push heroku main` pour déployer

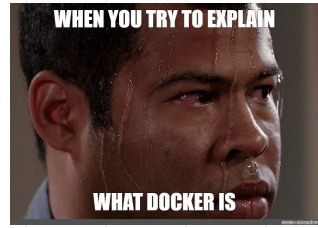
GCP (Google Cloud Platform)

- Installez Google Cloud SDK
- Créez un fichier `requirements.txt` pour les dépendances de votre application
- Créer un fichier `app.yaml`

```
runtime: python39
entrypoint: streamlit run app.py

handlers:
- url: /*
  script: auto
```

- Authentifiez-vous et déployez :
 - a. `gcloud auth login`
 - b. `gcloud config set project YOUR_PROJECT_ID`
 - c. `gcloud app deploy`
- Obtenez l'URL fournie après le déploiement ou via la console App Engine



Déploiement d'une application Streamlit

Docker

- Créer un **Dockerfile** et déployer sur n'importe quel serveur compatible Docker

```
# Utilisez une image Python comme image de base
FROM python:3.9-slim

# Définir le répertoire de travail dans le conteneur
WORKDIR /app

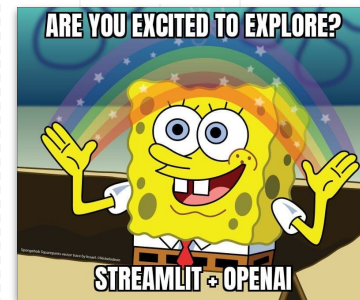
# Copier les fichiers de l'application dans le conteneur
COPY requirements.txt ./
COPY app.py ./

# Installer les dépendances depuis le fichier requirements.txt
RUN pip install --no-cache-dir -r requirements.txt

# Exposer le port sur lequel l'application Streamlit sera accessible
EXPOSE 8501

# Définir la commande pour lancer l'application Streamlit
CMD ["streamlit", "run", "app.py"]
```

Pour aller plus loin



- **Concepts supplémentaires :**

- **Composants personnalisés** (avec JavaScript et React : <https://docs.streamlit.io/library/components/create>)
- **LLM** (chat et LangChain : <https://docs.streamlit.io/develop/tutorials/lms>)
- Affichage des **Dataframes** (<https://docs.streamlit.io/develop/concepts/design/dataframes>)
- **Connexion** à des sources de données (AWS S3, BigQuery, MySQL, Snowflake, Tableau, etc. : <https://docs.streamlit.io/develop/tutorials/databases>)
- **App testing** (<https://docs.streamlit.io/develop/concepts/app-testing>)
- Exécution par **fragment** (<https://docs.streamlit.io/develop/tutorials/execution-flow>)
- Gestion des **secrets** : <https://docs.streamlit.io/develop/concepts/connections>

- Streamlit **Cheat Sheet** : <https://docs.streamlit.io/develop/quick-reference/cheat-sheet>
- La **roadmap** Streamlit : <https://roadmap.streamlit.app/>
- Des **tutoriels** : <https://docs.streamlit.io/develop/tutorials>

Streamlit API cheat sheet

This is a summary of the docs, as of Streamit v1.38.0

Install & Import

```
pip install streamlit

streamlit run first_app.py

# Import convention
>>> import streamlit as st
```

Command line

```
streamlit --help
streamlit run your_script.py
streamlit hello
streamlit config show
streamlit cache clear
streamlit docs
streamlit --version
```

Pre-release features

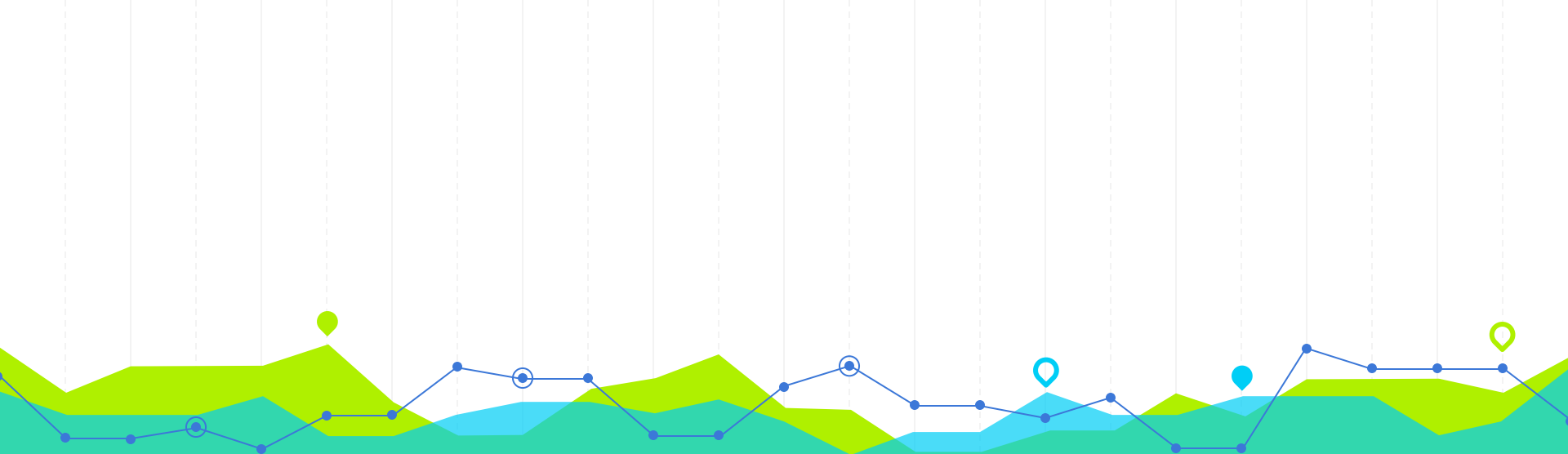
```
pip uninstall streamlit
pip install streamlit-nightly --upgrad
```

Learn more about experimental features

```
# Magic commands implicitly
# call str.write().
"_This_ is some **Markdown**"
my_variable
"dataframe", my_data_frame
```

Display interactive widgets

[illegible]



Le mot de la fin

Comment coder en Python pour de la production

Pour résumer

1. Organisation du code source :

- Utilisez une **structure de projet standard** telle que celle recommandée par [The Hitchhiker's Guide to Python](#).
- Utilisez `venv` ou `uv` pour gérer les **dépendances** de votre projet (`pyproject.toml`)

2. Contrôle de Version :

- Utilisez un système de contrôle de version comme **Git**
- Adoptez une convention de **nommage de branche** claire : `feature/`, `bugfix/`, `hotfix/`, etc.

3. Stratégie de Versioning :

- Adoptez la version sémantique ([Semantic Versioning](#)) :
 - MAJOR.MINOR.PATCH, où :
 - MAJOR : des changements incompatibles
 - MINOR : des changements compatibles (ajouts de fonctionnalités)
 - PATCH : des corrections de bugs compatibles
- Utilisez des **tags** Git pour marquer les versions de votre code.

Comment coder en Python pour de la production

Pour résumer

4. Tests et intégration continue :

- Écrivez des **tests unitaires** en utilisant des outils comme `pytest`.
- Configurez un outil d'**intégration continue (CI)** comme Jenkins, GitHub Actions ou GitLab CI

5. Automatisation du build :

- Utilisez des outils comme `setuptools` pour définir **comment votre package Python doit être construit**
- Considérez l'utilisation de `pipenv` ou `poetry` pour gérer à la fois les dépendances et le packaging

6. Distribution :

- Si vous développez une **bibliothèque**, envisagez de la **publier sur PyPI**
- Si vous développez une **application**, envisagez de la "containeriser" avec Docker ou de la distribuer en utilisant des outils comme `PyInstaller` ou `cx_Freeze`

Comment coder en Python pour de la production

Pour résumer

7. Documentation

- Documentez votre code à l'aide de **docstrings** et générez une **documentation** avec des outils comme [Sphinx](#)
- Pensez à la documentation externe, telle que les fichiers README, CHANGELOG, CONTRIBUTING, etc.

8. Maintenabilité

- Adoptez un **style** et une **qualité** de code cohérent avec des linters comme [flake8](#) et formateurs de code comme [autopep8](#)
- Gardez les **dépendances à jour**, avec des outils comme [dependabot](#) ou [pyup.io](#) pour vous aider à gérer les mises à jour

9. Retours :

- Utilisez des outils de **suivi des bugs** et de **gestion des tâches** pour organiser le travail et recueillir les commentaires
- Mettez en place des mécanismes pour recueillir les erreurs à l'exécution (par exemple, les **logs**, les **rapports d'erreurs**).

Une application Python bien construite doit être **bien organisée**, **bien testée**, correctement **versionnée** et facilement **distribuable**.
Adopter ces bonnes pratiques **dès le début** !

Comment être le meilleur



1. **Maîtrise avancée de Python** : connaître Python sur le bout des doigts, y compris ses subtilités
2. **Compréhension des structures de données et des algorithmes** : optimaux pour chaque situation
3. **Pratiques de codage solides** : écrire un code propre, lisible, efficace et maintenable (PEP 8)
4. **Expérience avec des cadres et des bibliothèques courants** : Flask, Pandas, Numpy, TF, etc.
5. **Compréhension des systèmes** : systèmes d'exploitation, des bases de données, des réseaux, etc.
6. **Connaissance des pratiques DevOps** : CI/CD, la gestion des conteneurs, Kubernetes, etc.
7. **Habiletés de débogage et de résolution de problème** : identifier, localiser et résoudre les problèmes dans le code
8. **Connaissance des tests** : tests unitaires, des tests d'intégration et des autres formes de tests
9. **Esprit critique** : revoir et d'évaluer la qualité du code, y compris le sien.
10. **Curiosité** : toujours chercher à apprendre et à se tenir au courant (veille) des dernières tendances
11. **Capacités de communication** : communiquer et vulgariser avec d'autres développeurs et clients
12. **Travail d'équipe** : capable de travailler efficacement en équipe
13. **Pensée adaptative** : capacité à s'adapter et à évoluer
14. **Connaissances interdisciplinaires** : avoir des connaissances métiers peut être bénéfique

La quête de la maîtrise est un voyage sans fin



Merci !